



ITAHARI
INTERNATIONAL
COLLEGE

Module Code & Module Title:

CS4051 Fundamentals of Computing

Assessment Weightage & Type:

70% Individual Coursework

Milestone Submission (Week 10)

Year and Semester:

2025 Autumn

Student Name: Anusha Bishwas

London Met ID: 25028734

College ID: NP05CP4A250031

Assignment Due Date: April 24, 2026

Assignment Submission Date: April 24, 2026

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1. Introduction	2
1.1 Aims and Objectives	2
1.1.1 Aim.....	2
1.1.2 Objectives	2
1.2 Technologies Used.....	2
Development Environment and Programming Tools.....	2
Flowchart Design Tool	3
2. Algorithm	4
3. Flowchart.....	5
4. Data Structures	6
4.1 Primitive Data Types.....	6
4.2 Collection Data Types.....	6
5. Conclusion	8
5.1 Issues Faced and Resolutions.....	8

Table of Figures

Figure 1: Visual Studio Code.....	2
Figure 2: Draw.io	3
Figure 3: Flowchart Representation	5
Figure 4: Primitive Data Types	6
Figure 5: List	7
Figure 6: Nested List	7

Table of Tables

Table 1:Primitive Data Types	6
Table 2:Collection Data Types	7
Table 3: Issues and Resolutions	8

1. Introduction

This project involves the development of a medicine inventory management system that reads medicine information from a text file and displays it in a clear and readable format. The system helps keep track of medicine stock including name, manufacturer, available quantity, pricing per tablet and per strip, and the number of tablets per strip. The program is the first milestone for the Fundamental of Computing coursework.

1.1 Aims and Objectives

1.1.1 Aim

The primary aim of this project is to develop a python program that reads, stores, and displays medicine inventory data from a text file and presents the information in a well-formatted display within the python shell and ensures the program handles errors without crashing.

1.1.2 Objectives

To achieve the stated aim, the following objectives were set:

- To read medicine information from a structured text file.
- To store data in appropriate python collections(list).
- To display information in a clear and well-structured readable format in the Python shell.
- To implement error handling for missing files or corrupted data.

1.2 Technologies Used

Development Environment and Programming Tools

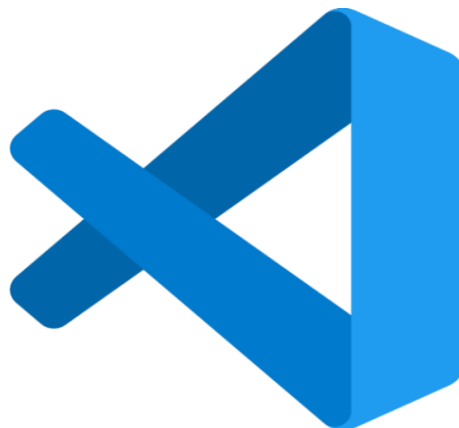


Figure 1: Visual Studio Code

This project was developed using Python 3 as the main programming language. It only uses basic built-in functions like `open()`, `strip()`, and `split()`, without any external libraries, so it's easy to run. All the coding, debugging, and testing were done using Visual Studio Code (VS Code) as the IDE.

Flowchart Design Tool

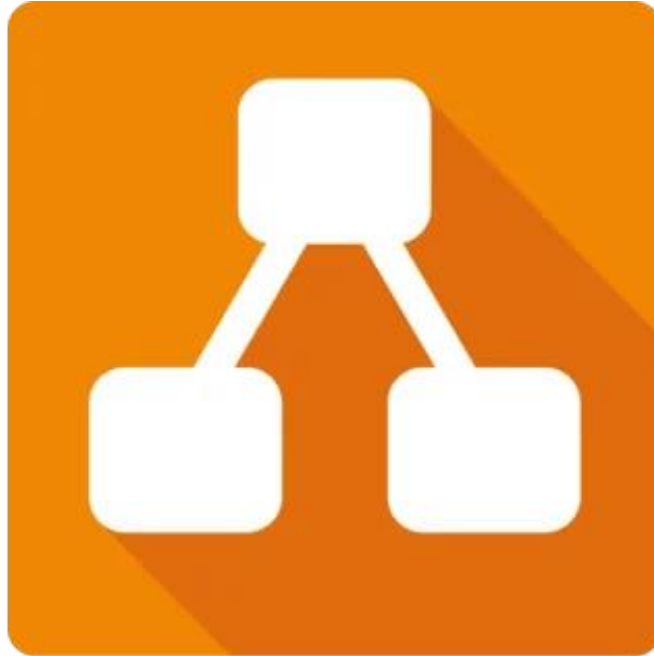


Figure 2: Draw.io

The flowchart for the algorithm was created using draw.io (also known as diagrams.net), a free tool that makes it easy to design and export flowcharts in formats like PNG, JPEG, and PDF.

2. Algorithm

The algorithm below shows the step-by-step logic used in the `load_and_display()` function. It starts by opening the text file, then goes through each line, splits the comma-separated values, and stores them in a list. Finally, it displays the data in a clear and readable format and also handles any file-related errors.

Step 1: START

Step 2: DEFINE function `load_and_display(medstore)`

Step 3: TRY:

- i. OPEN file in read mode
- ii. CREATE empty list called "medicines"
- iii. FOR EACH line in the file:
 - REMOVE whitespace using `strip()`
 - SPLIT line by comma into a list called "data"
 - APPEND "data" to "medicines" list
- iv. CLOSE the file
- v. DISPLAY column headers for the table
- vi. FOR EACH medicine in "medicines":
 - EXTRACT name, manufacturer, stock, price per tablet, price per strip, tablets per strip
 - DISPLAY total number of medicines loaded

Step 4: EXCEPT `FileNotFoundError`:

- i. DISPLAY error message for missing file

Step 5: EXCEPT any other Exception:

- ii. Display error message

Step 6: END TRY

Step 7: CALL `load_and_display("text.txt")`

Step 8: END

3. Flowchart

The flowchart shows how the program works step by step. It starts by opening a file, then reads each line and adds it to a list. After that, it displays the information in a clear format. If the file can't be found, it follows a separate path to handle the error instead of crashing.

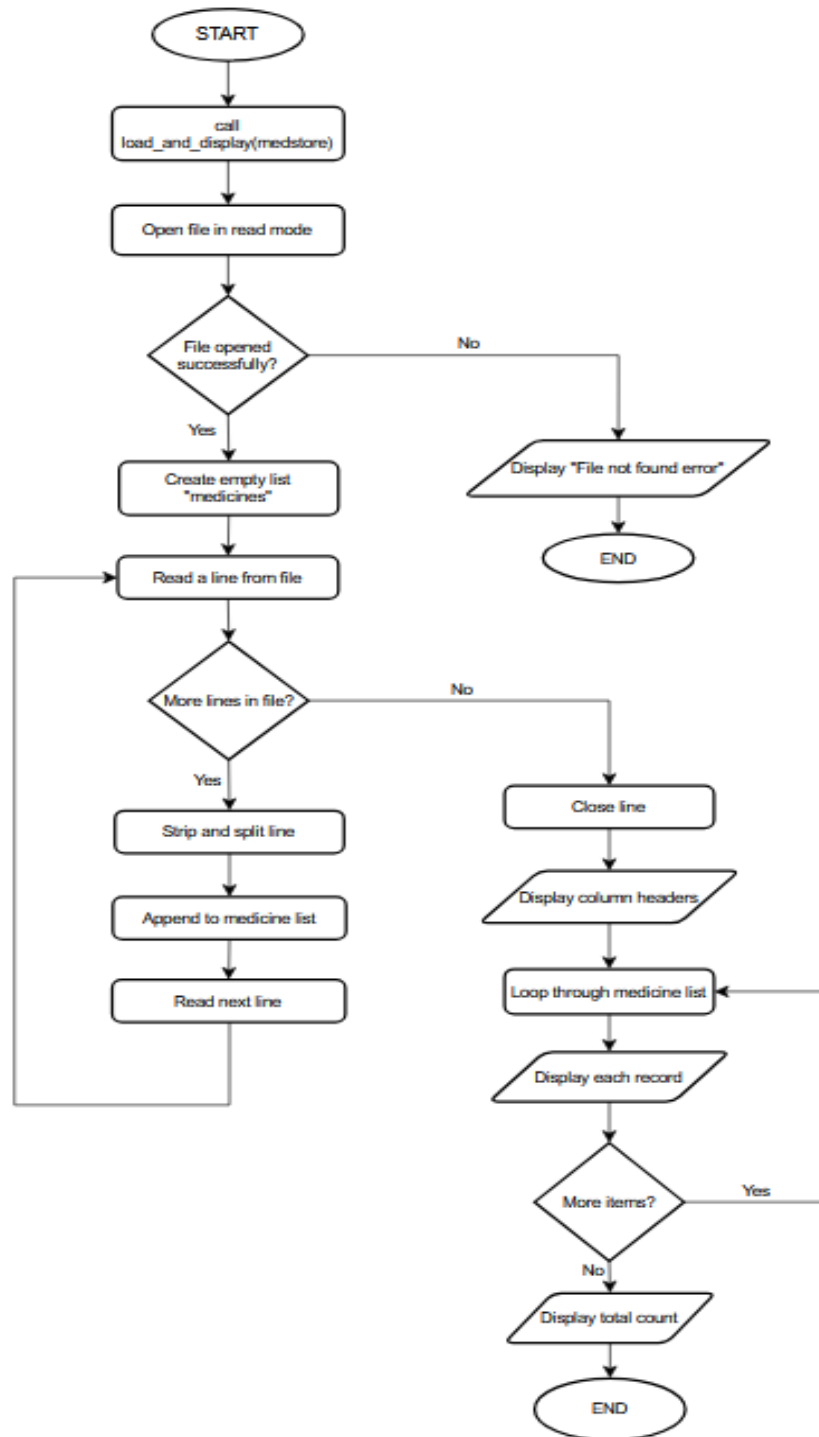


Figure 3: Flowchart Representation

4. Data Structures

The program mainly uses two main data structures. A list is used to store all the medicine records, and each record is another list containing six details: name, brand, stock, price per tablet, price per strip, and tablets per strip. Since every record follows the same format, there is no need for dictionaries or other complex structures.

4.1 Primitive Data Types

Primitive data types are simple types that store a single value.

Table 1: Primitive Data Types

Data Type	Description	Example from Code
String (str)	Used to store text values	"Cetirizine 10mg", "Quest", "800", "text.txt"
Integer (int)	Whole numbers without decimals	105 (used in "=" * 105), len(medicines)
Float(float)	Numbers with decimal points	Not used in this version, but would apply to prices like 4.50

Code Example of Primitive Types:

```
filename = "text.txt"           # String
separator_length = 105         # Integer
name = med[0].strip()          # String
```

Figure 4: Primitive Data Types

4.2 Collection Data Types

Collection data types are used to store multiple values in one place. In this program, lists are mainly used.

Table 2: Collection Data Types

Collection Type	Description	Example from Code
List	An ordered and changeable collection that can store multiple items	medicines = [] creates an empty list
Nested List	A list that contains other lists inside it	Each medicine record is stored as a list inside the main medicines list

Code Example of List:

```
# Creating an empty list
medicines = []

# Appending data to the list
medicines.append(data) # data is a list of 6 strings
```

Figure 5: List

Code Example pf Nested List:

```
# How the medicines list looks internally
medicines = [
    ["Cetirizine 10mg", "Quest", "800", "4", "35", "12"],
    ["Amoxicillin 500mg", "Nepal Remedies", "500", "12", "110", "15"],
    ["Pantoprazole 40mg", "Deurali-Janta", "600", "7", "60", "8"],
    ["ORS Sachet", "Time Pharma", "300", "20", "180", "12"]
]

# Accessing data from nested list
first_medicine_name = medicines[0][0]      # "Cetirizine 10mg"
second_manufacturer = medicines[1][1]      # "Nepal Remedies"
third_stock = medicines[2][2]              # "600"
```

Figure 6: Nested List

Lists were used as the main data structure because they are simple and easy to work with. Since the data comes from a comma-separated file, it fits naturally into a list of lists, where each inner list represents one medicine. Looping through a list makes it easy to go through and display all the data with a simple for loop, which makes the code easier to read.

5. Conclusion

Through this milestone, I learned some key basics of Python programming. I worked with file handling, including opening, reading, and closing files properly using `open()`, and understood why closing files matters. I also learned how to turn raw text data into structured information by splitting lines and cleaning them using `strip()` and `split(",")`.

I practiced error handling using try-except blocks to deal with problems like missing files, which makes the program more reliable. I also used f-strings to format output neatly and display data in a clean tabular format.

Finally, I learned the benefit of writing code in functions, which helps keep the program organized and reusable.

5.1 Issues Faced and Resolutions

During the development of this program, I faced a few challenges. The table below explains each problem and how it was fixed.

Table 3: Issues and Resolutions

Issues	Explanation	Resolution
Blank lines in text file	Empty lines can cause errors when splitting data	Used <code>strip()</code> and added a check to skip empty lines
Inconsistent spacing in output	Different text lengths make the table look uneven	Used f-string formatting (e.g. <code>:<30</code>) to align columns properly
File not found error	Program crashes if the file name is wrong or missing	Added try-except <code>FileNotFoundError</code> to show a friendly error message
Data stored as strings	Numbers like stock and price cannot be used for calculations	Kept as strings for display; can be converted using <code>int()</code> or <code>float()</code> if needed.
Hardcoded filename	File name was fixed inside the function, making it less flexible	Changed function to take filename as a parameter for reuse

This milestone met its goal of reading, storing, and displaying the medicine data. The program handles errors properly, shows the output in a neat table, and follows a clear algorithm. These basics will help in the next stage of the project.